A 60-Vertex Lower Bound for Cubic Bipartite Counterexamples to the Erdős–Gyárfás Conjecture

Julius Tranquilli
Independent researcher
jtranqs@gmail.com

29 July 2026
DOI: 10.5281/zenodo.21695514

Abstract

I give an exact exhaustive computation showing that every simple cubic bipartite graph on at most 58 vertices contains a cycle of length 4, 8, or 16. Consequently, any cubic bipartite counterexample to the Erdős–Gyárfás conjecture has at least 60 vertices. To my knowledge, this raises the established published lower bound applicable to this class from 30 to 60; relative to the most recent public minimum-degree-three computation, it raises the bound from 32 to 60. Equivalently, the computation newly excludes all fifteen admissible orders $30, 32, \ldots, 58$ beyond the published cubic frontier. Candidates are encoded as symmetric 3-uniform, 3-regular incidence structures and generated by a complete rooted restricted-growth procedure. Two separately written implementations cover side sizes 7 through 29, and a third covers side sizes 9 through 29. All agree on deterministic search-transcript hashes throughout their common ranges. In addition, a separately written streaming checker verifies a static full-range witness certificate in which every C_8- or C_{16}-based pruning decision carries an explicit positive witness. Source code, logs, certificates, and reproducibility materials accompany the paper.

Keywords. Erdős–Gyárfás conjecture; cubic bipartite graphs; prescribed cycle lengths; exhaustive generation; symmetric configurations; computer-assisted proof.

2020 Mathematics Subject Classification. Primary 05C38; Secondary 05C30, 68V05.

1 Introduction

The Erdős–Gyárfás conjecture asks whether every finite simple graph of minimum degree at least three contains a simple cycle whose length is a power of two [5]. In a simple graph the first relevant lengths are $4, 8, 16, 32, \ldots$. The conjecture remains open, although it is known for several restricted graph classes; examples include 3-connected cubic planar graphs [6], P_{10}-free graphs [8], and, with computer assistance, P_{13}-free graphs [7]. Recent work also gives strong structural restrictions on a minimal counterexample [4].

Numerically, the result raises the established published lower bound applicable to the cubic-bipartite class from $n \geq 30$ to $n \geq 60$, and raises the newest public computational bound from $n \geq 32$ to $n \geq 60$.

This paper concerns the finite-order frontier inside the class of simple cubic bipartite graphs. Its main result is deliberately stated in a stronger form than a bare verification of the conjecture.

Theorem 1 (Finite cubic-bipartite frontier)**.** *Every simple cubic bipartite graph G with*

$$|V(G)| \le 58$$

contains a simple cycle of length 4*,* 8*, or* 16*.*

Corollary 2. *Any simple cubic bipartite counterexample to the Erdős–Gyárfás conjecture has at least* 60 *vertices.*

Proof. Let L and R be the two bipartition classes of a counterexample G. Cubicity gives

$$3|L| = |E(G)| = 3|R|,$$

so $|L| = |R|$ and $|V(G)|$ is even. Theorem 1 excludes all possible orders through 58; the next possible order is 60. □

1.1 Literature chronology and the comparison baseline

Markström's exhaustive computation, published in 2004, established the customary cubic lower bound $n \ge 30$: a cubic counterexample has at least 30 vertices [9]. This is the last clearly published cubic finite-order result located in my audit.

There is a source discrepancy concerning a 2011 computation for bipartite graphs. The accessible proceedings abstract states a lower bound of 30 vertices [11]. Later papers attribute a bound of 32 to the same work; this attribution appears, for example, in the claw-free literature [12] and in Hu–Shen [8]. In contrast, Hegde, Sandeep, and Shashank report the value 30 [7]. I did not locate the underlying code or a public certificate, so I do not use the disputed value as an input to the proof.

A public SAT/SAT-Modulo-Symmetries artifact subsequently reported that all graphs of minimum degree at least three on at most 31 vertices contain a power-of-two cycle, yielding the general bound $n \ge 32$ [1]. The repository was created in June 2026, reached $n \ge 31$ on 18 June, and recorded the $n \ge 32$ result in commit `53502b0f` on 3 July 2026. It provides source code, exhaustive outputs, and multiple cross-checks, but no complete end-to-end LRAT-style certificate for all dynamically introduced constraints.

Accordingly, there are two useful comparisons:

Table 1: The two finite-order baselines used in this paper. "Increase" means the numerical change in the lower bound, not a count of newly excluded admissible orders.

Comparison source	Previous	Present	Increase	Newly excluded cubic-bipartite orders
Published cubic result (Markström, 2004)	$n \ge 30$	$n \ge 60$	30	$30, 32, \dots, 58$ (15 orders)
Public minimum-degree-three computation (3 July 2026)	$n \ge 32$	$n \ge 60$	28	$32, 34, \dots, 58$ (14 orders)

The relevant chronology depends on the comparison class. Within the published literature identified here, the preceding finite-order improvement specific to cubic graphs is Markström's 2004 computation. More recently, the public minimum-degree-three computation recorded on 3 July 2026 advanced the general bound to 32, which also applies to cubic bipartite graphs. The intervening 2011 bipartite record is ambiguous: the accessible primary abstract states 30, whereas later secondary sources attribute 32 to the same work.

Remark 3. Theorem 1 is slightly stronger than a finite-order verification of the conjecture: it does not use the possibility of a 32-cycle, even though such a cycle fits at the orders considered. One of the first three relevant power-of-two lengths is already forced.

1.2 Proof architecture

The proof has an ordinary mathematical part and a computational part. First, a cubic bipartite graph is represented as the incidence graph of a symmetric 3-uniform, 3-regular set system. In this representation, 4- and 8-cycles have simple incidence interpretations, and a newly created 16-cycle is equivalent to an old simple path of length 14. Second, a rooted restricted-growth rule is proved complete for all connected candidates. Finally, exact implementations enumerate this search tree and return zero completions for every side size through 29.

Conceptual reduction. An elementary cubic Moore-bound observation suggests a shorter route that is not used by the certified search. If a cubic bipartite graph on at most 58 vertices has no C_4, C_6, or C_8, then its girth is at least 10. The two nonbacktracking trees of depth four rooted at the endpoints of an edge would therefore contain

$$2(1 + 2 + 4 + 8 + 16) = 62$$

distinct vertices, a contradiction. Thus a hypothetical graph in the range of Theorem 1 with no C_4, C_8, or C_{16} must contain a C_6. Under the incidence translation, it is consequently enough to exclude connected linear symmetric v_3-configurations with $v \leq 29$ that contain a Berge triangle but no Berge cycle of length 4 or 8. The certified computation below proves the stronger result obtained by omitting the Berge-triangle hypothesis. This observation does not define or alter the retained search tree, counters, or certificate.

2 Incidence configurations and cycle translations

Let $G = (X, Y; E)$ be a connected simple cubic bipartite graph. As in the proof of Corollary 2, write

$$|X| = |Y| =: v, \qquad |V(G)| = 2v.$$

For each $y \in Y$, let $B_y = N_G(y)$ and regard $\mathcal{B} = (B_y)_{y \in Y}$ as an indexed family of three-element blocks on the point set X. Repeated triples are allowed at this stage: distinct vertices of Y remain distinct block indices even if they have the same neighborhood. The family has v indexed blocks, and each point belongs to exactly three of them.

Proposition 4 (Incidence translation)**.** *The neighborhood construction is a bijection, up to the natural relabelings, between connected simple cubic bipartite graphs with specified bipartition* (X, Y) *and connected* 3*-uniform,* 3*-regular incidence structures* $(X, (B_y)_{y \in Y})$ *having* v *points and* v *indexed blocks.*

Proof. The preceding construction gives the incidence structure. Conversely, make one graph vertex for each point and one for each block index y, and join x to y precisely when $x \in B_y$. Uniformity gives degree three on the block side, regularity gives degree three on the point side, and the two kinds of objects form a bipartition. Each membership is a Boolean relation, so there is at most one edge between a given point and block index even when two indexed blocks are equal. The two notions of connectedness are the same because an incidence walk is exactly a graph walk in the constructed bipartite graph. □

Definition 5. An indexed triple system is *linear* if blocks with distinct indices have at most one common point; equivalently, no unordered pair of points occurs in two indexed blocks.

Lemma 6 (C_4)**.** *The incidence graph contains a simple* 4*-cycle if and only if two distinct blocks contain the same pair of points.*

Proof. A 4-cycle in a bipartite incidence graph alternates as

$$x, B, x', B', x$$

with $x \neq x'$ and $B \neq B'$. Thus $x, x' \in B \cap B'$. Conversely, two blocks sharing distinct points x, x' give exactly this 4-cycle. □

By Lemma 6, it suffices after the first rejection test to work with linear triple systems. These are the symmetric combinatorial v_3-configurations of the configuration literature; their Levi graphs are exactly the cubic bipartite graphs of girth at least six [2, 3].

A Berge cycle of length $k \geq 3$ consists of distinct blocks $B_0, \ldots, B_{k-1}$ and distinct points $p_0, \ldots, p_{k-1}$ with $p_i \in B_i \cap B_{i+1}$, where indices are read modulo k. Equivalently, it is a simple C_{2k} in the Levi graph; in particular, a C_6 is a Berge triangle.

Lemma 7 (C_8)**.** *In a linear triple system, the incidence graph contains a simple* 8*-cycle if and only if there are four distinct blocks B_0, B_1, B_2, B_3 and four distinct points p_0, p_1, p_2, p_3 such that*

$$p_i \in B_i \cap B_{i+1} \qquad (i \bmod 4).$$

In other words, the blocks contain a Berge quadrilateral.

Proof. A simple 8-cycle alternates between four distinct point vertices and four distinct block vertices. Reading it cyclically gives the displayed incidences. Conversely, those incidences form the alternating closed walk

$$p_0, B_1, p_1, B_2, p_2, B_3, p_3, B_0, p_0.$$

The stipulated distinctness makes this walk a simple 8-cycle. Linearity ensures that a pair of consecutive blocks has at most one intersection point, so the test is unambiguous. □

Lemma 8 (Incremental C_{16} oracle)**.** *Let H be a partial incidence graph with no C_{16}, and add a new block vertex b adjacent to the three points in $B = \{x, y, z\}$. A new simple C_{16} is created if and only if H contains a simple path of length* 14 *between two distinct members of B.*

Proof. Any C_{16} appearing after the augmentation but not before it must contain the new vertex b. A simple cycle uses exactly two of the three edges at b. Delete b and those two cycle edges. What remains is a simple 14-edge path in H between the corresponding two distinct members of B.

Conversely, let P be a simple 14-edge path in H between two distinct points $u, w \in B$. The new vertex b does not occur on P. Adding the edges ub and bw closes P into a 16-edge cycle. Because P is simple and b is new, this cycle is simple. □

The statement allows one or two members of B to be newly introduced points. Such a point has degree zero in H, so it cannot be an endpoint of an old nontrivial path; the equivalence remains valid without a special case.

3 Complete rooted restricted-growth generation

3.1 Root normalization

Fix a point called 0. In a C_4-free candidate, the three blocks through 0 contain six distinct other points: if two of the blocks reused another point, Lemma 6 would give a C_4. The points can therefore be relabeled so that the rooted star is

$$\{0,1,2\}, \qquad \{0,3,4\}, \qquad \{0,5,6\}. \tag{1}$$

In particular, every C_4-free cubic incidence structure has $v \geq 7$.

After fixing (1), point 1 requires two further incident blocks. The first such block has one of three forms.

Lemma 9 (Root-stabilizer orbits)**.** *Up to a permutation that fixes* 0 *and* 1 *and preserves the rooted star in* (1)*, the lexicographically first additional block through point* 1 *has exactly one of the following representatives:*

$$\{1,3,5\}, \qquad \{1,3,7\}, \qquad \{1,7,8\}.$$

Proof. Of the two still-uninserted target blocks through 1, choose first one having the greatest number of points already present in the root star. This is the block that can be made lexicographically first, because every old label is smaller than every subsequently assigned label.

It cannot contain point 2, because the pair $\{1,2\}$ already occurs. Nor can it contain both points from $\{3,4\}$, or both from $\{5,6\}$, since either choice repeats a pair from the root star. Thus its two positions other than 1 contain: two old points, one from each of the other root pairs; one old point and one new point; or two new points.

The permitted root stabilizer fixes $0,1,2$, may interchange the rooted blocks $\{0,3,4\}$ and $\{0,5,6\}$, and may swap the two nonroot points within either block. It therefore sends the old entries in the three cases to $3,5$, to 3, or to no old entry, respectively. Assigning the first new labels as $7,8$ gives $\{1,3,5\},\{1,3,7\},\{1,7,8\}$. In the first case, any second two-old block uses the unused members of the two root pairs and is lexicographically later; in the second case, the old entry of any other one-old block can be given a larger root-star label; and a block with fewer old entries is later automatically. Hence the selected representative is compatible with the generator's lexicographic order. The three cases have different numbers of old points and so are distinct orbits. □

3.2 The augmentation rule

At a partial state, labels $0,\dots,m-1$ have been introduced. The state stores the ordered list of blocks, each point degree, and the set of point pairs already used. The next augmentation is chosen as follows.

(1) Choose the least introduced point p whose current degree is below three.

(2) Add the remaining blocks through p in lexicographic order. Write a proposed block as $\{p,q,r\}$, where $p<q<r$.

(3) Existing choices for q,r must have labels greater than p, degree below three, and must not repeat a used pair with p.

(4) A point encountered for the first time receives label m. If the same block introduces two points, they receive labels $m,m+1$. Thus $m+1$ may occur only together with m.

(5) Reject the proposal if it violates a degree or linearity constraint, or if it creates a C_8 or C_{16} according to Lemmas 7 and 8. Otherwise insert it and recurse.

The lexicographic lower bound for successive blocks through the same point prevents permutations of those blocks from generating separate branches. The remaining normalization is a restricted-growth convention, not a canonical labeling under the full graph isomorphism group.

Proposition 10 (Completeness). *For each v, every connected linear 3-uniform, 3-regular incidence structure on v points whose incidence graph has no C_8 or C_{16} occurs in at least one branch of the three-orbit restricted-growth search.*

Proof. Take such a structure and choose a root point. Normalize its three incident blocks as in (1). By Lemma 9, relabel the first remaining block through point 1 to one of the three orbit representatives.

I construct a branch while simultaneously assigning labels not yet fixed. Immediately before processing a point p, maintain the following induction invariant:

(i) every target block whose least-labelled point is less than p has been inserted;

(ii) all target blocks already inserted through p form a lexicographically initial segment of all target blocks through p; and

(iii) the introduced labels are exactly $0, \dots, m-1$, in first-occurrence order.

The normalized root and first block establish this invariant at $p = 1$. Let B be the first target block through p not yet inserted. Every target block containing p and a smaller point was inserted when that smaller point was processed, by (i). Consequently the other two points of B are greater than p.

Whenever B contains a point not encountered earlier, assign it the next unused label; if two new points occur, assign the two consecutive unused labels in either fixed order. The proposal is therefore among the generator's choices and is later than the preceding block through p. Inserting it preserves (ii) and (iii). The target is linear and has no C_8 or C_{16}, so no target block is removed by a rejection test. Once the degree of p reaches three, advance to the least unfinished introduced point. Then (ii) for p becomes (i) for the next point, so the invariant continues by induction.

It remains to see that every target point is eventually introduced. If the introduced vertices formed a proper subset closed under all incident blocks, the incidence graph would have no edge from that subset to its complement, contrary to connectedness. Thus the process introduces every point and eventually inserts all target blocks. The resulting labeled path is present in one of the search branches. □

Remark 11. Proposition 10 is a coverage statement, not a uniqueness statement. Multiple rooted labelings of one unrooted graph may survive. A zero-completion result needs only that every candidate occur at least once. Accordingly, none of the state counters below is described as a count of nonisomorphic graphs.

Lemma 12 (Leaf condition). *At a completed search leaf, the generated incidence graph is connected, simple, cubic, bipartite, and contains no C_4, C_8, or C_{16}. Conversely, every graph covered by Proposition 10 reaches such a leaf.*

Proof. At a leaf there are v introduced points, v triples, and every point has degree three. Each triple has size three, and pair uniqueness gives C_4-freeness by Lemma 6. Every new point was first introduced in a block containing an old point, so the incidence graph remains connected to the rooted star. The incremental rejection tests preserve the absence of C_8 and C_{16}. The converse is the conclusion of Proposition 10. □

4 Exact implementations and certified computational result

4.1 Search implementations and decision transcripts

The discovery implementation, `research/src/canonical_root_orbit_search.cpp`, tests a proposed block for a C_{16} by depth-first enumeration of simple paths. For each of the three point pairs in the block, it asks whether the old incidence graph has a simple path of exactly 14 edges. A visited-vertex array enforces simplicity, and reaching the target too early causes rejection of that path. Its C_8 oracle directly enumerates the three old blocks that, together with the proposal, could form a Berge quadrilateral.

The second separately written implementation, `research/src/independent_mitm_root_orbit_search.cpp`, uses a different exact formulation. A C_8 is detected as an old simple path of length 6 between two proposed points. For a C_{16}, the program enumerates simple 7-edge half-paths from each endpoint. Two halves are joined only when they have the same midpoint and their visited-vertex bit masks intersect exactly in that midpoint. This is equivalent to an old simple path of length 14.

A third separately written implementation, `research/src/vector_mitm_root_orbit_verifier.cpp`, repeats the meet-in-the-middle formulation. Both meet-in-the-middle implementations use dynamically sized vectors for the complete half-path lists and for the lists grouped by midpoint. Neither imposes a global or per-midpoint capacity, and no enumerated half-path is truncated.

The programs also emit a versioned deterministic transcript digest for each side size and root orbit. The serialization records every recursive state, including the ordered partial block family; every attempted candidate block; its accept/reject outcome and rejection class; and terminal and return markers. A small portable SHA-256 implementation is checked against the standard empty-string and `abc` test vectors before transcript reproduction. The two principal programs produce identical digests for $v = 7, \ldots, 29$; the third produces the same digests throughout its supported range $v = 9, \ldots, 29$. The agreed digests and event counts are retained in `research/results/transcript_hashes.tsv`. Thus the agreement is at the level of ordered search decisions, not merely aggregate counters.

The programs share the high-level restricted-growth scheme, root normalization, and mathematical completeness argument. They provide source-code and C_{16}-oracle cross-checks, but not independent reproductions in the ordinary academic sense and not independent proofs of Proposition 10. All accept/reject decisions use exact integers, arrays, and bit masks. Floating-point arithmetic appears only in elapsed-time reporting.

4.2 A full-range static witness certificate

The artifact includes `research/certificates/eg58_witness_certificates.zip`, its SHA-256 metadata, a certificate generator, and a separately written streaming checker. The bundle contains 66 proof streams: the one available normalized root orbit at $v = 7$, both available orbits at $v = 8$, and all three orbits at every $v = 9, \ldots, 29$.

For each stream, the checker reconstructs the normalized root, every partial block family, and every candidate block in deterministic restricted-growth order. A candidate that fails a degree or pair condition is rejected directly by the checker and consumes no certificate record. Every other candidate is represented by exactly one of the following records:

(i) three old-block indices whose intersections with the proposed block witness a Berge quadrilateral, hence a C_8;

(ii) an endpoint pair and seven old-block indices witnessing a simple old path of length 14, which the proposed block closes into a C_{16}; or

(iii) an expansion record, after which the checker recursively reconstructs the complete child subtree.

Only positive cycle claims require witnesses. The checker need not certify that an expanded candidate is cycle-free: expanding an additional forbidden branch only enlarges the tree being exhausted. It rejects any invalid witness, malformed or truncated stream, inconsistent counter, trailing data, or expanded branch reaching a complete configuration.

Proposition 13 (Soundness of the witness checker)**.** *If the streaming checker accepts a certificate stream for a normalized root orbit at side size v, then that orbit contains no completed restricted-growth leaf.*

Proof. The checker itself regenerates every candidate at every visited state in the deterministic order prescribed in Section 3.2. Degree and pair failures are recomputed directly. A record of type (i) is accepted only after the checker verifies the four distinct blocks and four distinct intersection points required by Lemma 7. A record of type (ii) is accepted only after it verifies a simple 14-edge path between the stated endpoints, so Lemma 8 makes the rejection sound. A record of type (iii) is accepted only after the entire child stream has been checked recursively.

Induct on the recursive call tree. At each state, every candidate is either rejected for a condition verified by the checker or its complete child subtree is verified. The checker rejects immediately if a completed configuration is reached. Its end-of-stream and trailing-data checks ensure that no generated candidate or subtree can be silently omitted or ignored. Therefore acceptance closes every branch in the orbit and certifies that no completed leaf occurs there. □

All 66 streams verify, their per-side-size counters equal Table 2, and every completion count is zero. At $v = 29$, the three streams contain witnesses for all 34,098,891 C_8 rejections and all 12,934,970 C_{16} rejections. The complete deterministic ZIP is 20,545,969 bytes. A supplementary compatibility stream reproduces the semantic counters of the former $v = 16$, C_4/C_8-only text certificate.

Certificate trust boundary. The static bundle removes the production search programs and their cycle oracles from the certificate-checking trust base. It still relies on the compact checker and on the mathematical root-normalization and restricted-growth completeness argument in Proposition 10. It is not an LRAT proof or a proof-assistant formalization.

4.3 Range and counter semantics

The exhaustive range is

$$7 \leq v = |X| = |Y| \leq 29,$$

which corresponds to graph orders 14 through 58. Side sizes below seven are disposed of by the root-star argument: a cubic C_4-free incidence graph would require the root plus six distinct points.

For each side size, *states* counts recursive search entries; *attempted* counts proposed triples; *pair/C_4* counts degree, pair-repetition, and linearity rejections; the next two columns count the C_8-

and C_{16}-oracle rejections; and *completions* counts fully cubic leaves. Timing is excluded because it has no logical role.

Table 2: Fresh exact restricted-growth search counts regenerated from the included source. The graph order is $2v$.

v	states	attempted	pair/C_4	C_8	C_{16}	completions
7	1	1	0	1	0	0
8	2	6	1	5	0	0
9	5	40	9	29	0	0
10	10	78	13	58	0	0
11	44	418	91	286	0	0
12	95	1,377	267	1,018	0	0
13	165	3,073	517	2,394	0	0
14	268	5,294	795	4,234	0	0
15	448	9,664	1,422	7,789	8	0
16	751	18,853	2,603	15,450	52	0
17	1,233	34,415	4,356	28,737	92	0
18	1,904	59,427	7,005	50,429	92	0
19	2,801	99,989	11,132	85,147	912	0
20	4,052	162,708	17,126	134,721	6,812	0
21	5,971	254,881	25,440	198,617	24,856	0
22	8,967	404,844	38,624	291,952	65,304	0
23	14,276	712,704	65,454	478,545	154,432	0
24	24,666	1,413,328	123,413	912,651	352,601	0
25	45,106	2,907,902	241,277	1,849,740	771,782	0
26	82,499	5,997,887	470,746	3,838,495	1,606,150	0
27	153,469	12,667,279	943,843	8,194,975	3,374,995	0
28	282,344	25,587,480	1,817,786	16,772,786	6,714,567	0
29	531,193	51,088,500	3,523,449	34,098,891	12,934,970	0

Historical counters. The larger $v = 7, \dots, 28$ counters from the initial research package are exactly reproduced by the same search in an unreduced-root mode, before quotienting the first augmentation by the root stabilizer. Appendix B records the check. Table 2 is the sole full-range table used in the proof.

At the frontier, the three root-stabilizer branches have the following separate totals.

Table 3: Side size $v = 29$ (graph order 58), split by the three root-stabilizer orbits.

orbit	states	attempted	pair/C_4	C_8	C_{16}	completions
1	3,286	286,790	22,001	141,678	119,826	0
2	23,607	2,167,611	148,504	1,489,941	505,560	0
3	504,300	48,634,099	3,352,944	32,467,272	12,309,584	0
Total	531,193	51,088,500	3,523,449	34,098,891	12,934,970	0

In particular, for $v = 29$ the computation visits 531,193 states, attempts 51,088,500 blocks, and produces no completion. The exact integer totals and all root-orbit transcript hashes agree across the implementations supporting those orbits.

Proposition 14 (Exhaustive computation)**.** *For every side size $v = 7, \ldots, 29$, the complete restricted-growth search has zero completions. The two principal implementations agree on every integer counter and deterministic root-orbit transcript hash for this entire range. The third implementation agrees on every counter and transcript hash throughout $v = 9, \ldots, 29$. A separately written streaming checker also accepts a static witness stream for every available normalized root orbit in the full range.*

Proof. This is the code-dependent statement established by the supplied source files, result tables, retained logs, certificate streams, and reproduction scripts. The repository-root `SHA256SUMS` records the tracked release files. Running `research/scripts/reproduce_all_v7_v29.py` rebuilds the two principal programs for each side size and requires an exact diff of their counter tables. Running `research/scripts/reproduce_transcript_hashes.py` checks the SHA-256 implementation, rebuilds every implementation at every supported side size, requires equality of each common per-orbit transcript and event count, and checks each side-size sum against the primary counter table. Running `research/scripts/reproduce_v29_all_orbits.py` rebuilds all three programs, executes each root orbit at $v = 29$, and requires exact agreement of the six integer counters and transcript hashes. Finally, `research/scripts/reproduce_witness_certificates.py` compiles the streaming checker, verifies all 66 full-range certificate streams, and requires their per-side-size totals to equal Table 2. Proposition 13 converts those accepted streams into a certificate of zero completed leaves. □

Certified finite lemma. *Every connected linear* 3*-uniform,* 3*-regular hypergraph on at most* 29 *points that contains a Berge triangle also contains a Berge cycle of length* 4 *or* 8*.*

Proof. For fewer than 7 points, linearity at any point would require the six other points occurring in its three incident blocks to be distinct, which is impossible. For $7 \leq v \leq 29$, a counterexample would correspond to a completed branch of the restricted-growth search by Proposition 10 and Lemma 12, contradicting Proposition 14. The argument does not use the Berge-triangle hypothesis, so the certified computation proves the stated lemma as a special case of its stronger conclusion. □

Computer-assisted proof of Theorem 1. Suppose G is a simple cubic bipartite graph on at most 58 vertices with no C_4, C_8, or C_{16}, and choose a connected component H. Every component of a cubic graph is cubic, so H is connected, simple, cubic, bipartite, and has at most 58 vertices. By Proposition 4, H gives a connected symmetric 3-uniform, 3-regular incidence structure of side size $v = |V(H)|/2 \leq 29$. The absence of C_4 makes the structure linear by Lemma 6. If $v < 7$, the root-star argument is already a contradiction. If $7 \leq v \leq 29$, Proposition 10 and Lemma 12 place the structure at a completed search leaf, while Proposition 14 says there is no such leaf. This contradiction proves the theorem. □

5 Cross-checks and positive controls

5.1 Decision-transcript agreement and fresh reruns

The two production implementations agree for $7 \leq v \leq 29$. At $v = 9, \ldots, 29$, all three source files reproduce the same per-orbit counters and complete decision-transcript hashes. At $v = 7, 8$, the two implementations supporting those sizes also agree. During preparation of this report on 29 July 2026, the full-range counter script, the three-implementation transcript script, and the full frontier script were rerun on an ARM64 macOS system using Apple Clang 21.0.0. The fresh integer tables and every common per-orbit digest agreed. The full-range static certificate checker also accepted all 66 witness streams and reproduced the zero completion counts. The fresh outputs are retained under `research/logs/full_range/` and `research/logs/v29/`; the digest table is `research/results/transcript_hashes.tsv`.

5.2 Set-level comparison with `genbg`

The cycle-oracle agreement does not by itself test coverage of the restricted-growth tree. For a generator-level check, the auxiliary program `research/src/restricted_growth_c4_census.cpp` retains the same root and restricted-growth augmentation rules but disables the C_8 and C_{16} filters. For each $v = 7, \ldots, 13$, it emits every completed connected linear symmetric v_3-configuration as a graph6 Levi graph. The outputs are canonically labelled while preserving the two color classes, deduplicated, and compared byte-for-byte with nauty 2.9.3 `genbg` output generated with exact degree three and at most one common neighbor on the block side [10].

Table 4: Set-level generator comparison. The second column counts rooted and labelled restricted-growth leaves before deduplication; the third is the common number of color-preserving canonical graph6 records.

v	Rooted/labelled leaves	Canonical configurations
7	1	1
8	4	1
9	44	3
10	496	10
11	7,840	31
12	136,575	229
13	2,337,152	2,036

The canonical files and their SHA-256 hashes are stored under `research/results/genbg_crosscheck/`; the reproduction script requires exact equality of the actual graph6 sets, not merely equality of their cardinalities. The canonical counts also agree with the published census of Betten, Brinkmann, and Pisanski [2]. Because the `genbg` side uses a substantially different generator, this overlap checks restricted-growth coverage through $v = 13$. It does not replace the completeness proof for $v = 14, \ldots, 29$.

5.3 Certificate robustness tests

The certificate checker is deliberately strict about the proof-stream boundary. In addition to accepting the 66 unmodified streams, the supplied regression tests present malformed, truncated, trailing, counter-tampered, and witness-tampered streams. The checker rejects each such mutation rather than silently accepting a prefix, trusting stored counters, or ignoring invalid witness data. These negative tests exercise the parser and verification logic; the mathematical soundness of an accepted stream is the content of Proposition 13.

5.4 Positive controls

Negative exhaustive output is useful only if the rejection oracles are also shown to accept and classify known valid partial structures. The artifact contains 128 symmetric 19_3 configurations whose incidence graphs avoid C_4 and C_8 but contain C_{16}. Two algorithmically different checks were applied:

(a) NetworkX 3.6.1 elementary-cycle enumeration lists all simple cycles through length 16;

(b) an exact subset-state transfer dynamic program separately decides the existence of cycles of lengths $4, 8$, and 16.

Both methods classify all 128 inputs as

$$C_4\text{-free}, \qquad C_8\text{-free}, \qquad C_{16}\text{-containing}.$$

A supplementary permutation-model check decomposes the first incidence graph into three perfect matchings, normalizes one matching to the identity, and obtains the same cycle decisions by reduced color-word simulation. These are positive controls for cycle detection; they are not an additional exhaustive proof of the frontier.

6 Scope, limitations, and reproducibility

6.1 What has and has not been established

The result establishes the exact statement in Theorem 1. It does not prove the full Erdős–Gyárfás conjecture and does not exclude an order-60 counterexample. Side size 30 lies outside the verified range, so no bound of 62 is claimed.

The generator is complete for nonexistence but is not nauty/Traces-style isomorph-free generation; rooted duplicates may remain. The implementations share the same mathematical completeness argument, so a mistake in Proposition 10 would not be detected by counter or transcript agreement alone. The static checker reconstructs the restricted-growth tree rather than trusting a production generator, but necessarily implements the same enumeration schedule; its certificate therefore does not independently prove Proposition 10. The most valuable external checks would therefore be:

1. an independent human audit, or proof-assistant formalization, of the root-orbit and restricted-growth completeness proofs;
2. extension of the set-level `genbg` comparison beyond $v = 13$, or a second complete generator based on normalized permutation pairs; and
3. a substantially different full-range encoding, such as SAT/SMS with LRAT-capable proof logging.

The novelty claim is similarly qualified. The 60-vertex lower bound appears new relative to the public-source audit through 29 July 2026, but unpublished computations, private censuses, or uncatalogued code may exist.

6.2 Artifact availability

The preprint is archived on Zenodo at `DOI: 10.5281/zenodo.21695514`. The archived Zenodo record is version `v1.0.0`. The public computational artifact, including source code, witness certificates, verification programs, logs, and reproduction instructions, is available from the accompanying GitHub repository. The immutable public review artifact is release `v1.0.0-rc1`; its annotated Git tag records the corresponding full commit SHA. The repository-root `SHA256SUMS` covers every tracked snapshot file other than the manifest itself. `README.md`, `REPRODUCTION.md`, and `PROVENANCE.md` describe the layout, commands, and research provenance. The certificate directory contains the full-range witness bundle, detached digest, metadata, and byte-level format specification. The generator and streaming checker are retained under `research/src/`; checked-in tests construct the stream mutations at run time.

7 Conclusion

The incidence translation and complete restricted-growth generation reduce the finite cubic-bipartite problem to an exact search over side sizes at most 29. Every branch terminates without a completion, two distinct C_{16} implementations agree across the full range, and all three programs agree on decision-level transcripts throughout their common range. The full-range static witness certificate separately checks every C_8- and C_{16}-pruned branch through an explicit positive witness. Subject to the explicitly stated computer-assistance qualification, this proves that every simple cubic bipartite graph on at most 58 vertices contains a 4-, 8-, or 16-cycle. The lower bound for any cubic bipartite counterexample is therefore 60.

Acknowledgements and AI disclosure

The author thanks the developers and maintainers of the computational tools used in the verification workflow. OpenAI ChatGPT materially assisted the initial research process, including proposing computational and structural approaches, developing portions of the incidence-based search and verification code, running and interpreting computations, drafting preliminary mathematical arguments and the proof architecture, and conducting an initial literature audit.

OpenAI Codex subsequently assisted with reorganizing and auditing the artifact, reproducing computations, implementing additional cross-checks, resolving the historical generation-mode discrepancy, removing fixed meet-in-the-middle path stores, implementing the decision-transcript audit, designing and implementing the full-range witness-certificate generator and streaming checker, and drafting and editing the manuscript and documentation. Codex generated or modified portions of the manuscript and auxiliary code and invoked the reported reproduction commands in the author's workspace.

A custom closed-source coding/formalization harness built on a fork of Codex was also used during the later research and verification work. It provided an agentic environment for portions of the coding, mathematical formalization, artifact audit, and tool-driven checks. The harness is not included in the public artifact and is not treated as independent evidence.

Julius Tranquilli selected the problem and claimed result, directed the scope of the work, supplied the original dated package, and accepts responsibility for the final claims, code, computations, citations, and text. The computations ran in the author's workspace, in many cases through AI tool interfaces. The retained code, logs, exact outputs, certificates, and mathematical arguments—not an AI assertion—are the evidence reported here. Neither the AI systems nor the closed-source harness is an author or part of the evidentiary trust base. A fuller activity-level record, including the limits of file-level attribution for the early package, is provided in `PROVENANCE.md`.

A Restricted-growth pseudocode

The following pseudocode suppresses data-structure details but preserves the logical branch conditions. The three calls differ only in the normalized first block.

```
SEARCH(v, first_block):
    blocks := [{0,1,2}, {0,3,4}, {0,5,6}, first_block]
    introduced := 7, 8, or 9 according to first_block
    EXTEND(point = 1, previous = first_block)

EXTEND(point, previous):
    count.states += 1
```

```
    while point < introduced and degree(point) = 3:
        point += 1
        previous := none

    if point = v:
        if number_of_blocks = v:
            count.completions += 1
        return

    if point >= introduced or number_of_blocks >= v:
        return

    choices := eligible old labels greater than point
    append the next unused label, if available
    append the following unused label, if available

    for each ordered pair q < r from choices:
        require that a second fresh label occurs only with the first
        require {q,r} to be lexicographically later than previous
        count.attempted += 1
        B := {point,q,r}

        if B violates degree, pair uniqueness, or linearity:
            count.pair += 1
            continue
        if B closes a C8:
            count.c8 += 1
            continue
        if the old graph has a simple 14-path
           between two points of B:
            count.c16 += 1
            continue

        assign consecutive labels to fresh points in B
        insert B
        EXTEND(point, previous = (q,r))
        remove B and undo fresh labels
```

B Historical generation-mode check

The initial research package contained larger $v = 7, \ldots, 28$ state and rejection totals. Their generation mode has been identified exactly: the search starts directly from the fixed root star (1), before quotienting the next block through point 1 into the three root-stabilizer orbits of Lemma 9. It therefore traverses symmetric copies at the first augmentation while applying the same degree, pair, C_8, and C_{16} rules.

Both principal programs expose this historical mode as `-unreduced-root`. The script `research/scripts/verify_unreduced_root_counts.py` reproduces every integer entry in the canonical table `research/results/unreduced_root_counts.tsv` with both implementations. This diagnostic table is not primary evidence.

C Reproducibility summary

Exact commands and expected outputs are maintained in `REPRODUCTION.md`; the root file `SHA256SUMS` is the complete release manifest. Table 5 records one local run of each principal command. Measurements used a 10-core Apple M5 MacBook Pro with 16 GB RAM, macOS 26.4, Apple Clang 21.0.0, and Python 3.11.8. The `genbg` comparison used nauty 2.9.3. Times and memory are approximate and have no logical role. The `genbg` timing is for a clean run including the pinned nauty download, configuration, and build.

Table 5: Local reproduction resources. Peak RSS is rounded to the nearest MiB.

Command	Wall time	Peak RSS
`make verify-v29`	62 s	141 MiB
`make verify-full`	99 s	114 MiB
`make verify-transcripts`	153 s	117 MiB
`make verify-certificate`	16 s	127 MiB
`make generate-certificates`	78 s	127 MiB
`make verify-positive`	6 s	39 MiB
`make verify-genbg`	58 s	524 MiB
`make verify-unreduced`	78 s	111 MiB

References

[1] Arjun Balaji. Erdős–Gyárfás Conjecture for Minimum-Degree-Three Graphs. Public GitHub repository, 2026. The bound $n \geq 32$ was recorded in commit `53502b0f` on 3 July 2026; accessed 29 July 2026. https://github.com/ArjunBalaji79/erdos-gyarfas-min-degree-3.

[2] Anton Betten, Gunnar Brinkmann, and Tomaž Pisanski. Counting symmetric configurations v_3. *Discrete Applied Mathematics*, 99(1–3):331–338, 2000. https://doi.org/10.1016/S0166-218X(99)00143-2.

[3] Marko Boben. Irreducible (v_3) configurations and graphs. *Discrete Mathematics*, 307(3–5):331–344, 2007. https://doi.org/10.1016/j.disc.2006.07.015.

[4] Avery Carr. Every Minimal Counterexample to the Erdős–Gyárfás Conjecture Is Predominantly Cubic, 2026. arXiv:2605.22844. https://arxiv.org/abs/2605.22844.

[5] Paul Erdős. Some Old and New Problems in Various Branches of Combinatorics. *Discrete Mathematics*, 165–166:227–231, 1997.

[6] Christopher Carl Heckman and Roi Krakovski. Erdős–Gyárfás Conjecture for Cubic Planar Graphs. *Electronic Journal of Combinatorics*, 20(2):P7, 2013.

[7] Anand Shripad Hegde, R. B. Sandeep, and P. Shashank. Erdős–Gyárfás Conjecture on Graphs without Long Induced Paths, 2025. arXiv:2410.22842, version 2, 11 February 2025. https://arxiv.org/abs/2410.22842.

[8] Zhiquan Hu and Changlong Shen. The Erdős–Gyárfás Conjecture Holds for P_{10}-Free Graphs. *Discrete Mathematics*, 347(12):114175, 2024.

[9] Klas Markström. Extremal Graphs for Some Problems on Cycles in Graphs. *Congressus Numerantium*, 171:179–192, 2004. Publication record: https://urn.kb.se/resolve?urn=urn:nbn:se:umu:diva-19969.

[10] Brendan D. McKay and Adolfo Piperno. Practical Graph Isomorphism, II. *Journal of Symbolic Computation*, 60:94–112, 2014. https://doi.org/10.1016/j.jsc.2013.09.003.

[11] Pouria Salehi Nowbandegani and Hossein Esfandiari. An Experimental Result on the Erdős–Gyárfás Conjecture in Bipartite Graphs. In *14th Workshop on Graph Theory: Colourings, Independence and Domination*, 2011. Accessible proceedings abstract states a lower bound of 30; later sources sometimes attribute 32. http://www.cid.uz.zgora.pl/2011/files/AbstractsPdf/Salehi.pdf.

[12] Pouria Salehi Nowbandegani, Hossein Esfandiari, Mohammad Hassan Shirdareh Haghighi, and Khodakhast Bibak. On the Erdős–Gyárfás Conjecture in Claw-Free Graphs. *Discussiones Mathematicae Graph Theory*, 34(3):635–640, 2014. A later source attributing the bipartite bound 32 to the 2011 work.